Chronos Job Platform Overview

What It Is

Chronos combines a React + Bootstrap console with Spring Boot scheduler and worker services to orchestrate async jobs across a single stack. Together, the UI workflows, validated REST API, Kafka pipeline, and persistence layers cover creation, execution, and monitoring without stitching disparate tools.

Who It's For

Operators and developers who run Chronos async workloads and need scheduling, visibility, and remediation in one place.

What It Does

- Web console delivers signup/login, job creation (delayed / one-time / cron), activity feed, tracked status, and worker execution visibility for Chronos workloads (job-console-ui README).
- Admin tools for bulk email/notification payloads, failed-jobs view, and saved request drafts keep operators in control (job-console-ui README).
- Scheduler exposes a single POST /v1/jobs contract with schedule-type validation and a handler registry covering SEND_EMAIL, SEND_NOTIFICATION, RETRY_PAYMENT, ORDER_EXPIRY, HOTEL_NIGHT_AUDIT, and DATA_SYNC (job-scheduler-service README).
- Scheduler persists jobs in PostgreSQL via JPA, leases runs via Redis, enforces rate limits/security, and publishes job execution messages to Kafka (application.yml + JobExecutionPublisher).
- Worker consumes the job-execution Kafka topic, picks handler implementations for email, notification, payment, order, hotel, sync, news, and report flows, and records executions in a Postgres store (job-worker-service handlers + JobExecutionConsumer + JobExecutionController).
- Worker also produces failed-job CSV reports, pushes realtime updates back to the console via REST, and exposes HTTP endpoints for execution history/failed lists (FailedJobsReportService + ConsoleRealtimePushService + JobExecutionController).

How It Works

- Chronos UI proxies /api/scheduler/* to http://localhost:8087 and /api/worker/* to http://localhost:8088 while running on Vite (job-console-ui README).
- Scheduler REST API (/v1/jobs) validates, stores, and updates jobs and leases, then publishes execution payloads onto the job-execution Kafka topic with JobExecutionPublisher (JobController + JobExecutionPublisher).
- Kafka topic job-execution is consumed by JobExecutionConsumer, delegated to JobExecutionService, and dispatched through the handler registry for downstream endpoints, emails, notifications, sync jobs, and news feeds (job-worker-service/consumer + service + handler packages).
- Worker records execution outcomes in Postgres, exposes /v1/executions endpoints for listing and failed jobs, writes CSV reports, and uses failure backdoors + realtime push services to inform the scheduler/console of trouble (JobExecutionController + FailedJobsReportService + ConsoleRealtimePushService + WorkerFailureBackdoorService).

How to Run

1. `cd /Users/mihir/Documents/job-console-ui && npm install && npm run dev:notifications && npm run dev` (Vite on http://localhost:5173 with notification server on 5174).
2. `cd job-scheduler-service && mvn spring-boot:run` with DB_URL/DB_USERNAME/DB_PASSWORD pointing to a PostgreSQL database (application.yml).
3. `cd job-worker-service && mvn spring-boot:run` with DB_URL/DB_USERNAME/DB_PASSWORD, KAFKA_BOOTSTRAP_SERVERS, and any MAIL_* or HTTP endpoint env vars the handlers need (application.properties).